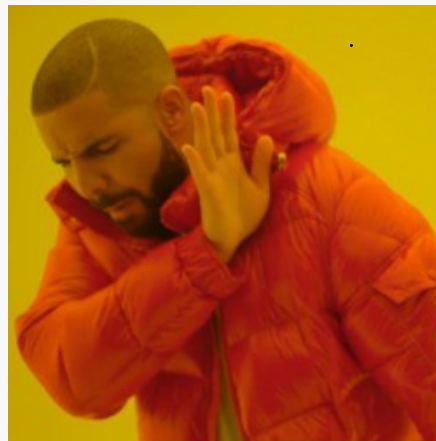


Massively Parallel Computing

[anshium.github.io](https://github.com/anshium)



Doing a simple
BFS in $O(N)$ time
on a single machine.

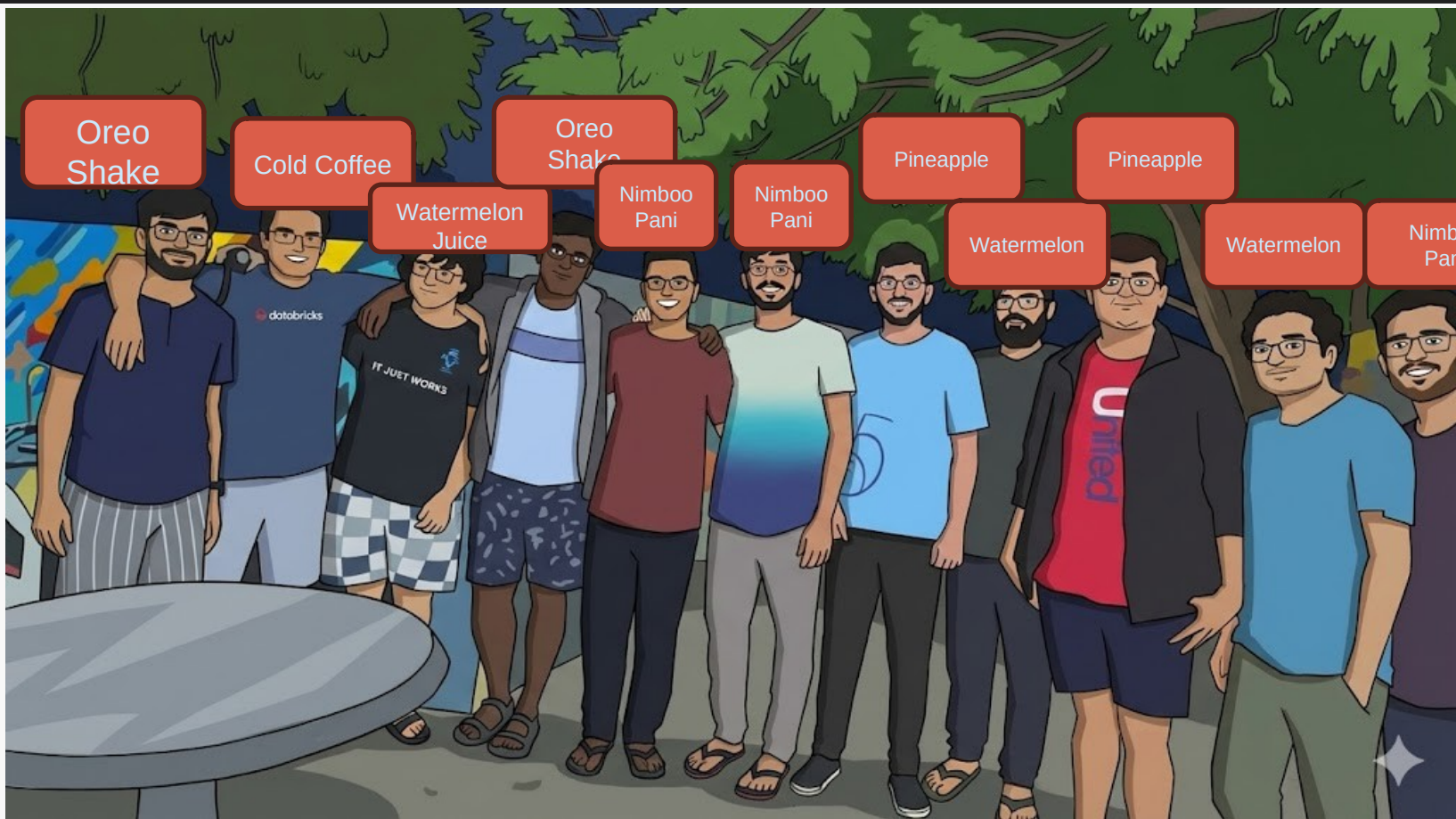


Spending 40
hours implementing
Borůvka's
algorithm
to save 3 rounds
of communication



**Imagine it
is 11:55 PM
and you
all are at
JC.**





Mr. Dinesh Yadav

Oreo
Shake

Cold Coffee

Oreo
Shake

Watermelon
Juice

Nimboo
Pani

Nimboo
Pani

Pineapple

Pineapple

Watermelon

Watermelon

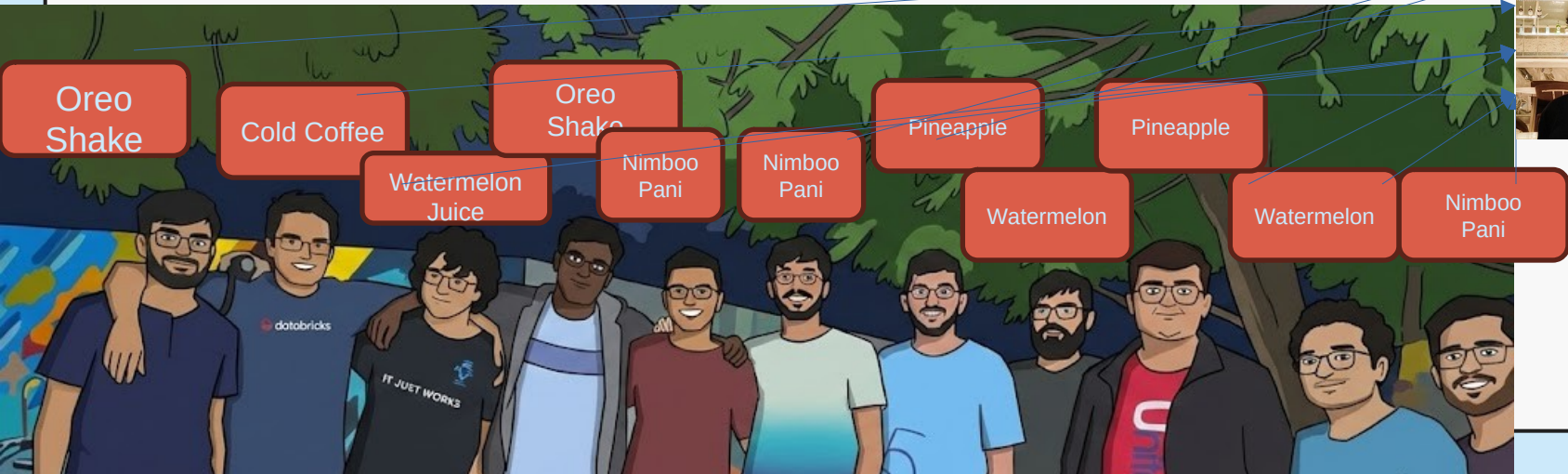
Nimboo
Pani



The Role of a Combiner

Mr. Dinesh Yadav

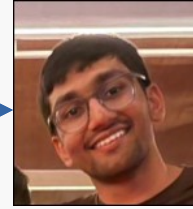
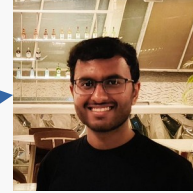
(Oreo Shake, 2),
(Cold Coffee, 1)
(Watermelon Juice, 3)
(Nimboo Pani, 3)
(Pineapple, 2)



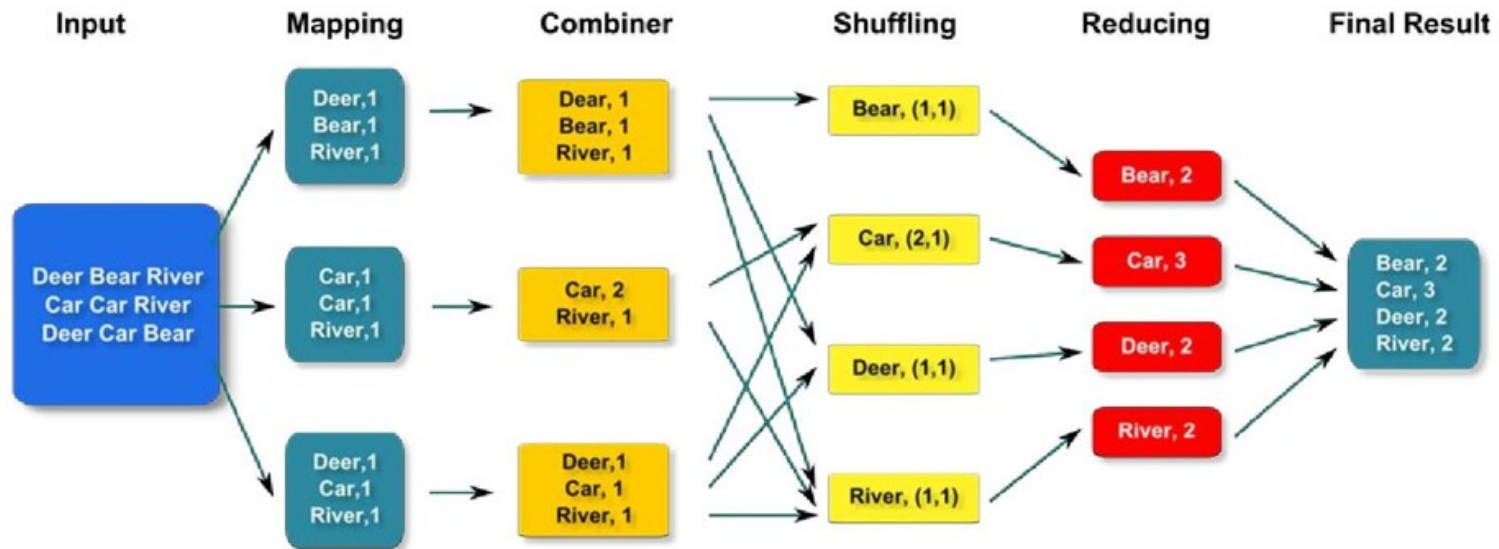
Multiple Combiners

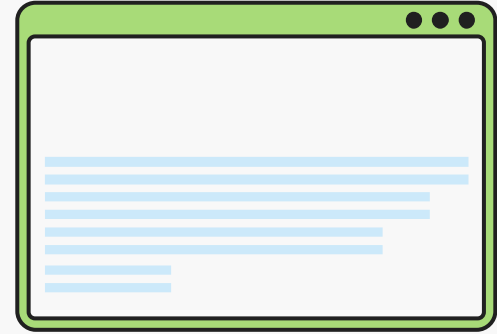
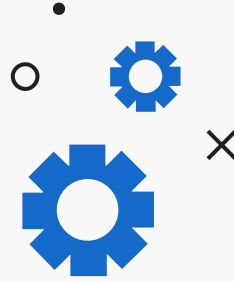
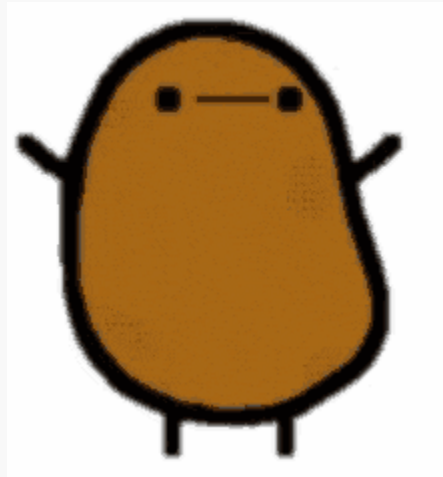


Multiple Combiners



Mr. Dinesh Yadav





Question 3
- Option 1

Remember this meme?



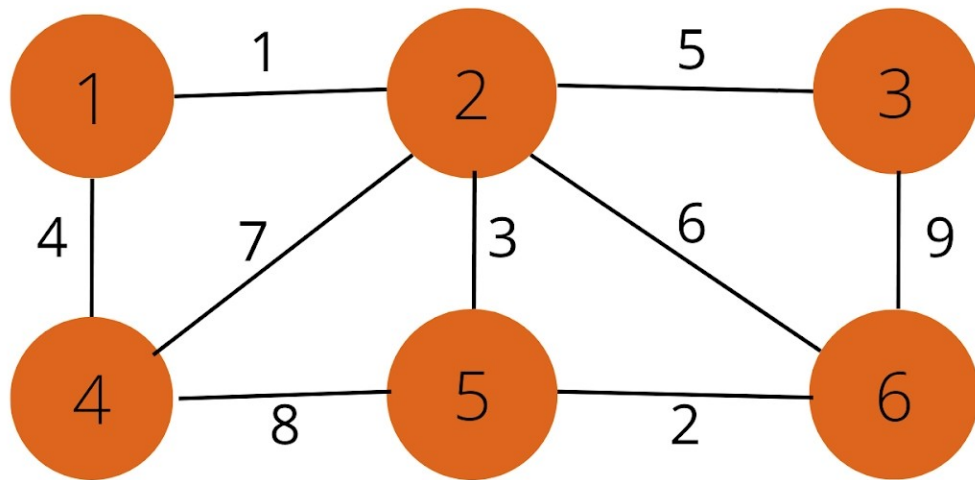
Doing a simple
BFS in $O(N)$ time
on a single machine.



Spending 40
hours implementing
Borůvka's
algorithm
to save 3 rounds
of communication

**This is a fairly
simple
algorithm to
understand**





```
// Step 1: Find Local Minimums
// Prepare an array to store the cheapest edge leaving each component
local_min_edges = array(size=num_nodes, init=INFINITY)

for edge (u, v, weight) in local_edges:
    root_u = find_component(u)
    root_v = find_component(v)

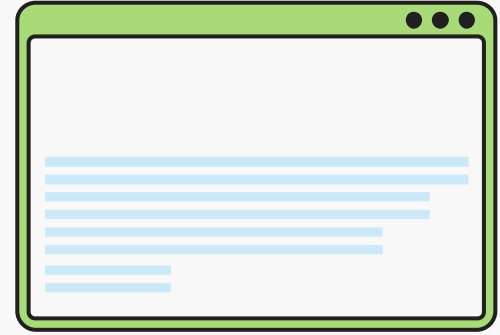
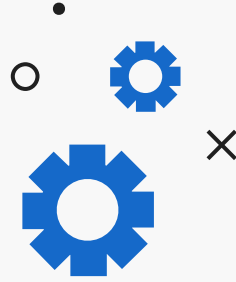
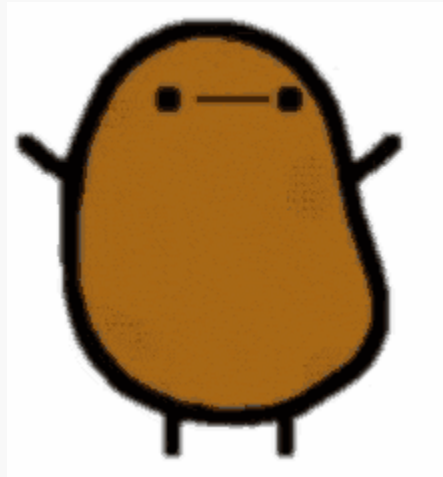
    if root_u != root_v:
        # Check if this is the cheapest edge for component root_u
        if weight < local_min_edges[root_u].weight:
            local_min_edges[root_u] = edge
        # Check for component root_v
        if weight < local_min_edges[root_v].weight:
            local_min_edges[root_v] = edge
```

```
// Step 2: Global Reduction (The "Communication" phase of MPC)
// Use MPI_Allreduce with MPI_MINLOC operation to find the global cheapest edge
// for every component across all ranks.
global_min_edges = MPI_Allreduce(local_min_edges, MPI_MIN)

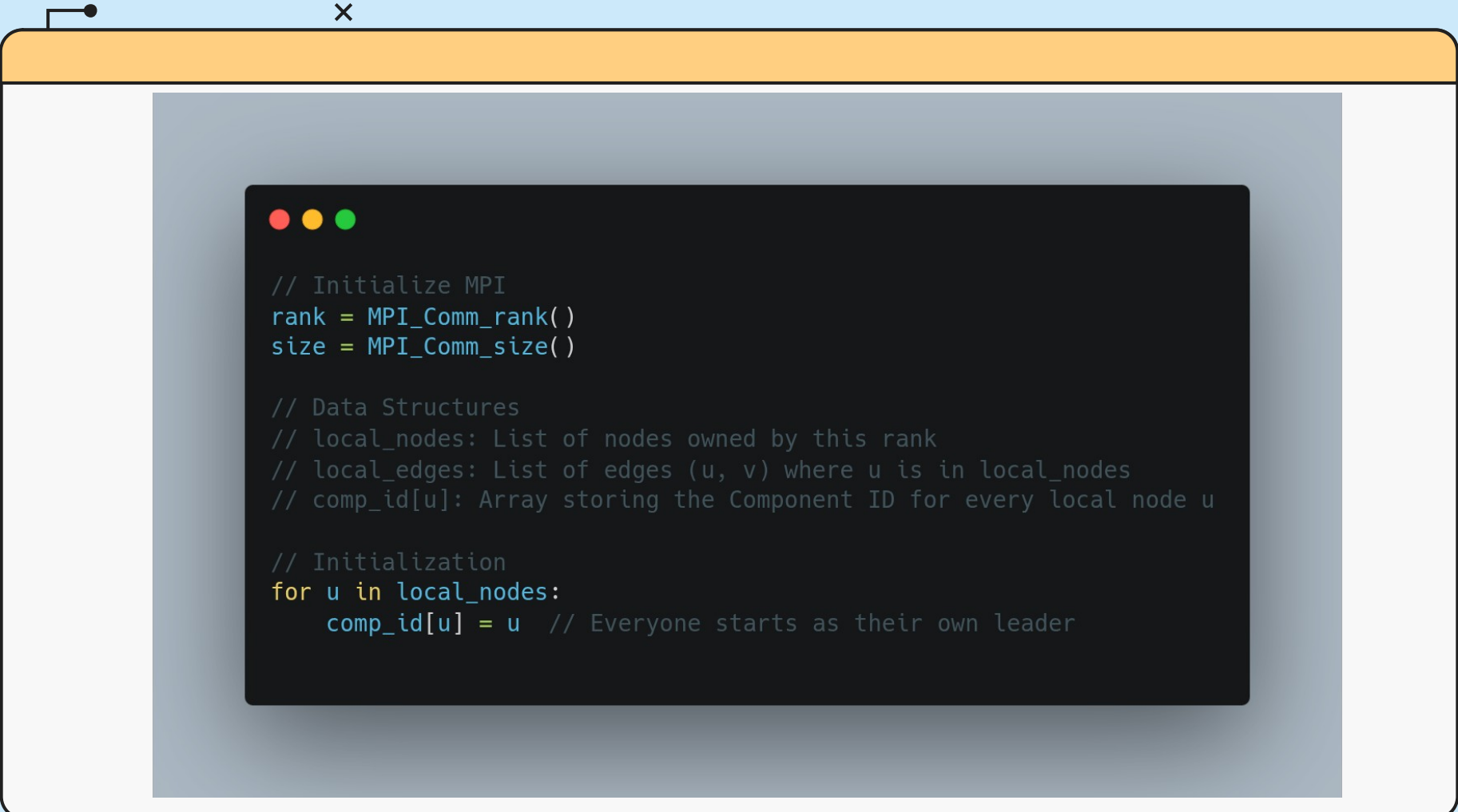
// Step 3: Merge Components (CPU phase)
did_merge = False
for i in 0 to num_nodes:
    edge = global_min_edges[i]
    if edge != INFINITY:
        u, v = edge
        union_sets(u, v) # Update component_id array
        add_weight_to_MST(edge.weight)
        did_merge = True

// Broadcast updated component IDs to all ranks so they have the new view
MPI_Bcast(component_id)

if not did_merge:
    break // Done
```



Question 3
- Option 2



```
// Initialize MPI
rank = MPI_Comm_rank()
size = MPI_Comm_size()

// Data Structures
// local_nodes: List of nodes owned by this rank
// local_edges: List of edges (u, v) where u is in local_nodes
// comp_id[u]: Array storing the Component ID for every local node u

// Initialization
for u in local_nodes:
    comp_id[u] = u // Everyone starts as their own leader
```

```
converged = False
```

```
while (not converged):  
    local_change = False
```

```
    // --- STEP 1: COMMUNICATION (Exchange IDs) ---
```

```
    // We need to send our current comp_id[u] to all neighbors v.
```

```
    // If v is on a different Rank, we use MPI_Isend / MPI_Recv.
```

```
    // Prepare buffers for boundary nodes (nodes connected to other  
    ranks) send_buffer = prepare_messages_for_neighbors(comp_id)
```

```
    // Asynchronous exchange to prevent deadlock
```

```
    MPI_Isend(send_buffer...)
```

```
    MPI_Recv(recv_buffer...)
```

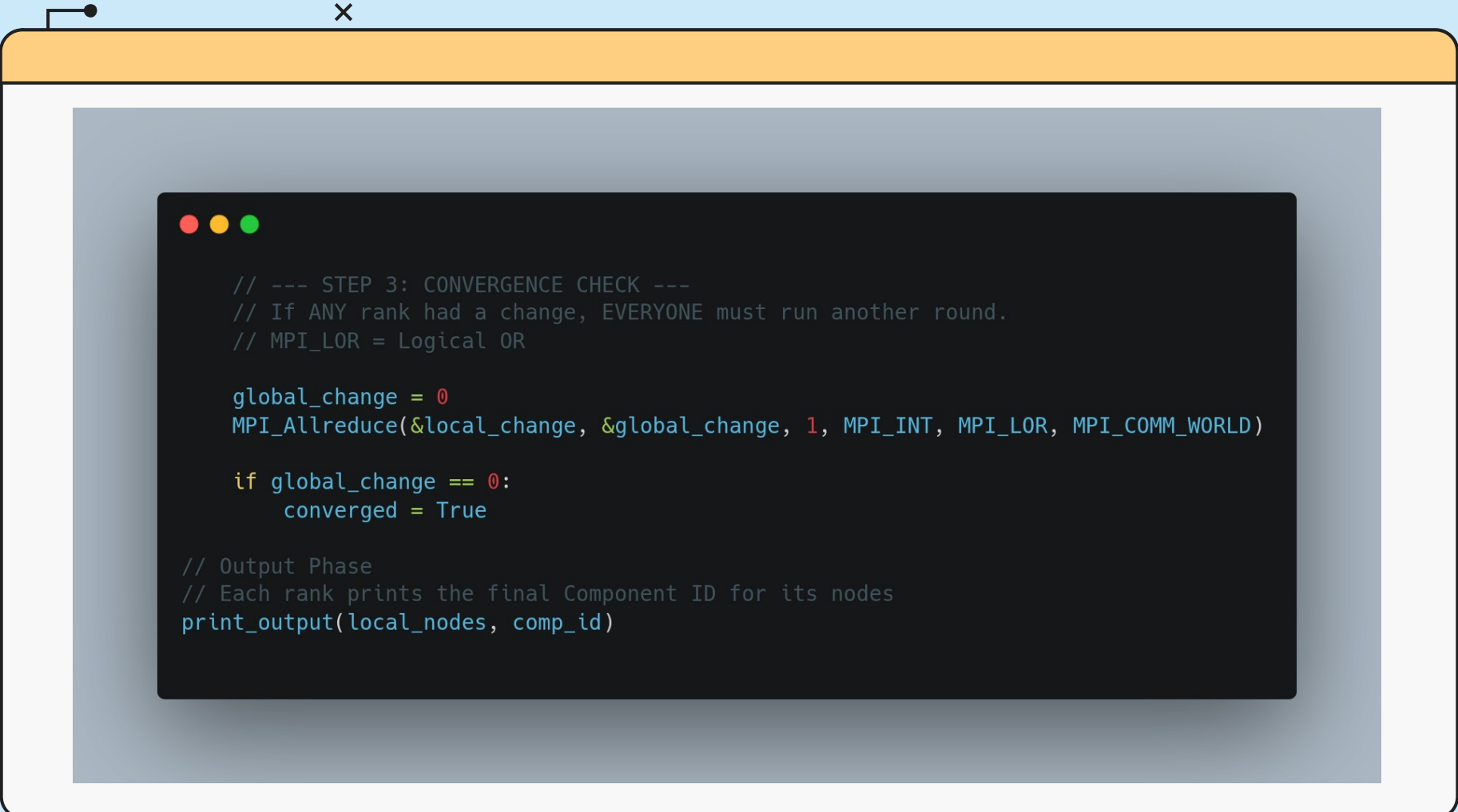
```
    MPI_Waitall(...)
```

```
// --- STEP 2: LOCAL COMPUTATION (Relaxation) ---  
// Update component IDs based on received info
```

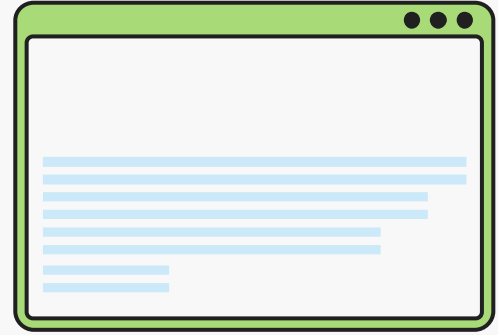
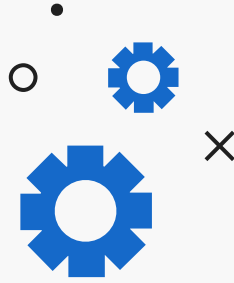
```
for (u, v) in local_edges:  
    neighbor_id = get_id_from_buffer(v) // Get v's ID from local mem or recv_buffer
```

```
    // The Core Logic: The smaller ID wins  
    if neighbor_id < comp_id[u]:  
        comp_id[u] = neighbor_id  
        local_change = True
```

```
    // Also update local edges internally (if v is also local)  
    if is_local(v) and comp_id[u] < comp_id[v]:  
        comp_id[v] = comp_id[u]  
        local_change = True
```



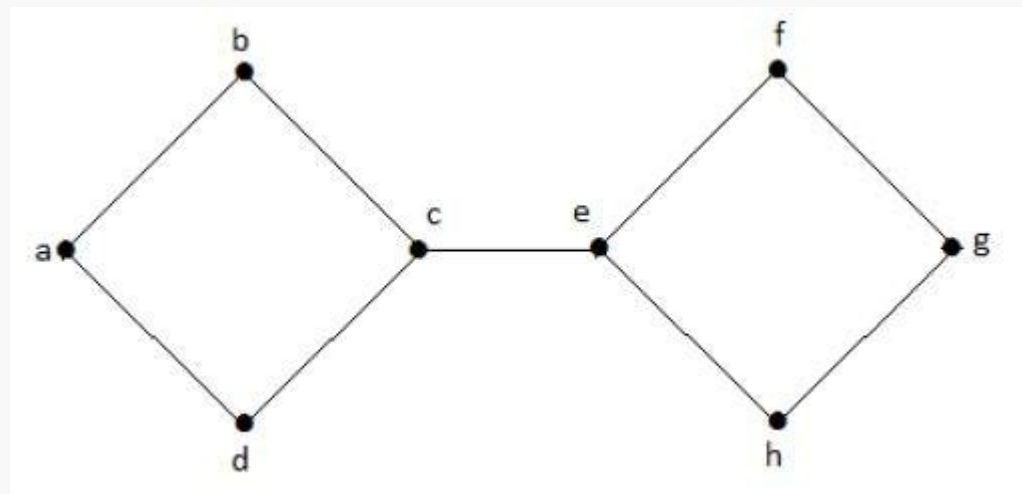
```
// --- STEP 3: CONVERGENCE CHECK ---  
// If ANY rank had a change, EVERYONE must run another round.  
// MPI_LOR = Logical OR  
  
global_change = 0  
MPI_Allreduce(&local_change, &global_change, 1, MPI_INT, MPI_LOR, MPI_COMM_WORLD)  
  
if global_change == 0:  
    converged = True  
  
// Output Phase  
// Each rank prints the final Component ID for its nodes  
print_output(local_nodes, comp_id)
```

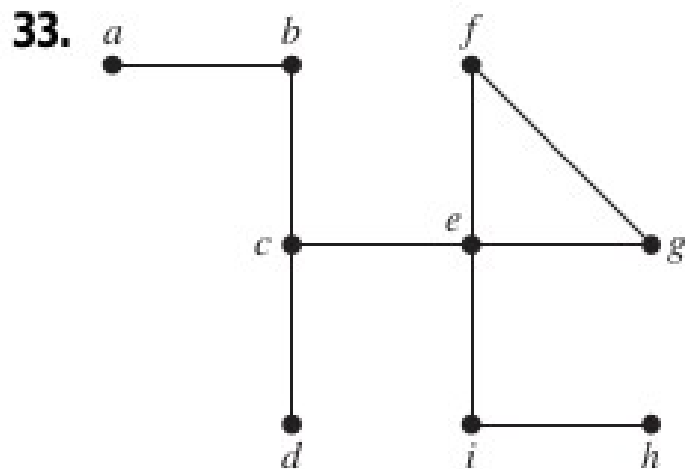
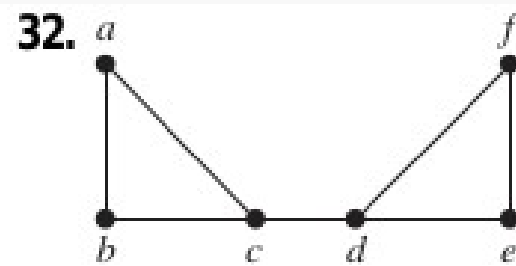
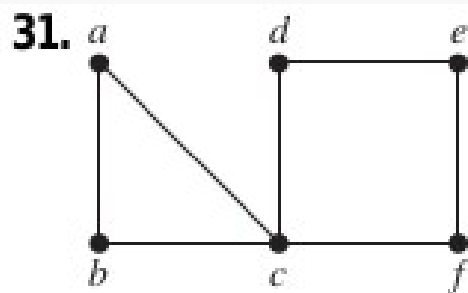


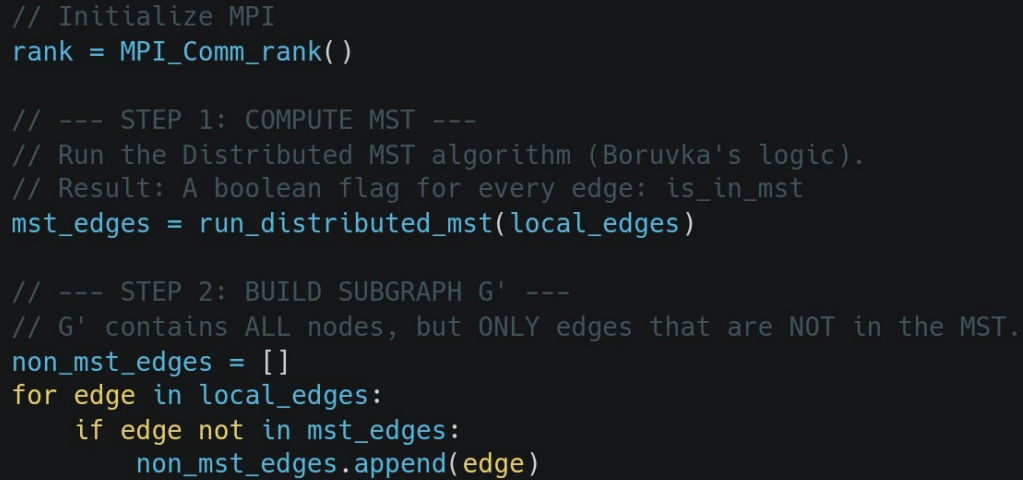
Question 3
- Option 3

**This has the
simplest
pseudo-code
of all three**





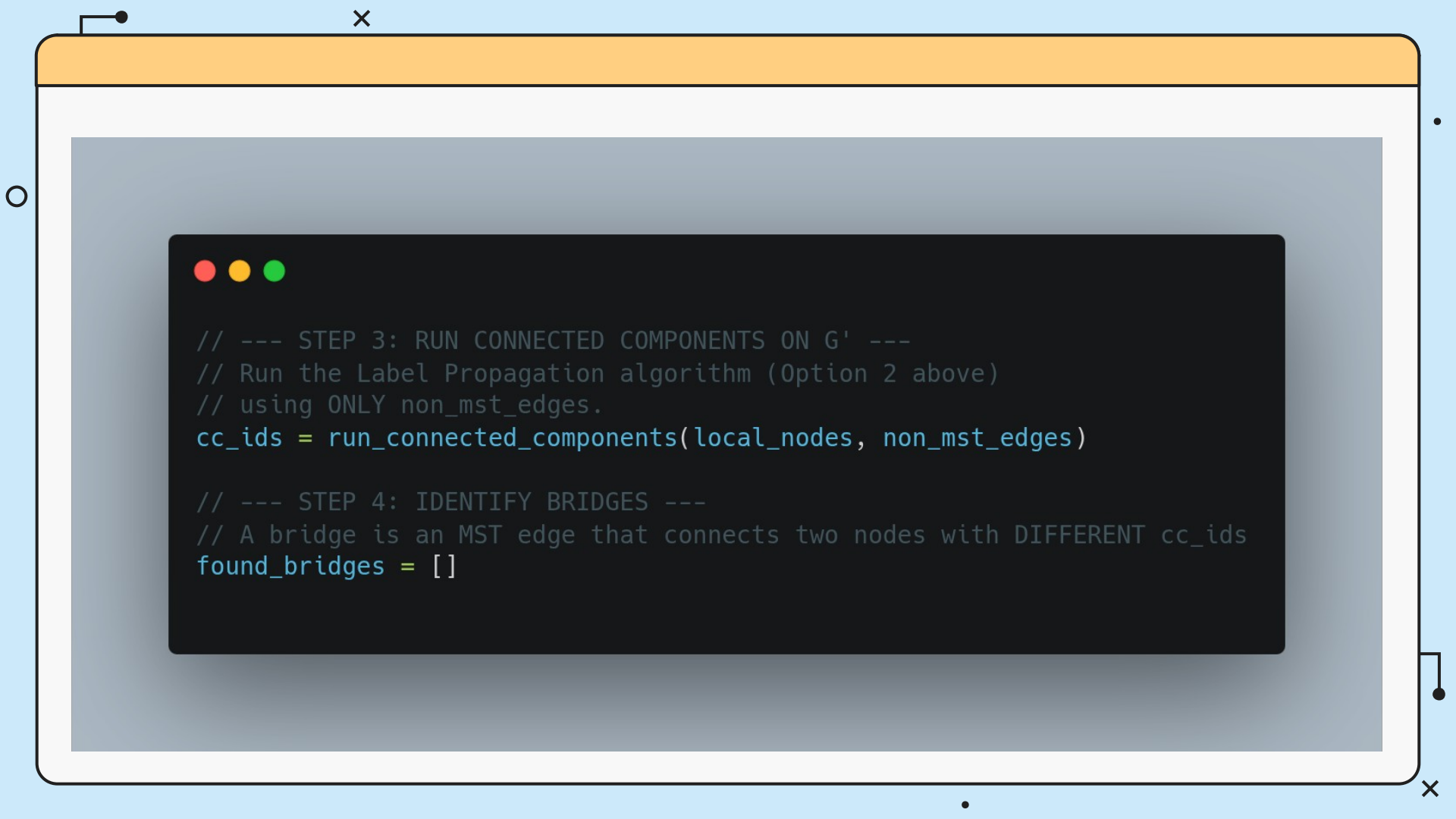




```
// Initialize MPI
rank = MPI_Comm_rank()

// --- STEP 1: COMPUTE MST ---
// Run the Distributed MST algorithm (Boruvka's logic).
// Result: A boolean flag for every edge: is_in_mst
mst_edges = run_distributed_mst(local_edges)

// --- STEP 2: BUILD SUBGRAPH G' ---
// G' contains ALL nodes, but ONLY edges that are NOT in the MST.
non_mst_edges = []
for edge in local_edges:
    if edge not in mst_edges:
        non_mst_edges.append(edge)
```



```
// --- STEP 3: RUN CONNECTED COMPONENTS ON G' ---  
// Run the Label Propagation algorithm (Option 2 above)  
// using ONLY non_mst_edges.  
cc_ids = run_connected_components(local_nodes, non_mst_edges)  
  
// --- STEP 4: IDENTIFY BRIDGES ---  
// A bridge is an MST edge that connects two nodes with DIFFERENT cc_ids  
found_bridges = []
```

```
for edge(u, v) in mst_edges:
    // We need the CC_ID of u and v.
    // Since edges might span ranks, we might need one communication step here
    // to ensure we know the CC_ID of the remote neighbor.

    id_u = cc_ids[u]
    id_v = get_remote_cc_id(v) // MPI_Sendrecv if v is remote

    if id_u != id_v:
        // The nodes are disconnected in G', meaning the MST edge
        // was the only link. It is a bridge.
        found_bridges.append((u, v))

// Output Phase
print_bridges(found_bridges)
```